Self-Hosted Garden Weather Station

System design and interface specification for Emmas Court Community Garden weather station

Revision History

Date	Editor	Change
3/09/2026	Brahm Schultz	Initial creation
3/09/2026	Brahm Schultz	Adding AWS backend spec

Revision History.... 2
1. Scope and design goals.... 4
2. Final recommended architecture.... 4
3. Hardware bill of materials.... 5
4. Antenna and connector notes for the RAK nodes.... 6
5. Interface specification: every device-to-device link.... 6
6. Data models and packet formats.... 8
    6.1 Tempest UDP packets consumed by the Pico.... 8
    6.2 Pico to Meshtastic payload format.... 8
    6.3 Home gateway parsing model.... 8
7. Power architecture.... 8
8. Home gateway design.... 9
9. Bench bring-up plan before adding the Pico.... 9
10. Pico integration plan.... 10
11. Known implementation notes from the current bench test.... 10
12. AWS side (intentionally lightweight for now).... 11
    12.1 Backend Overview.... 11
    12.2 Functional Requirements.... 11
    12.3 Recommended AWS Services.... 12
        1. API Gateway HTTP API.... 12
        2. AWS Lambda.... 12
        3. Amazon DynamoDB.... 13
        4. CloudWatch.... 14
13. Open items and recommended next actions.... 14

Recommended build in one sentence: Use a Tempest hub on a private garden Wi-Fi LAN, read the hub's local UDP broadcast on a Pico W, send compact weather packets over UART to a garden Meshtastic node, relay them over LoRa to a home Meshtastic node, then connect the home node to a Raspberry Pi over USB for forwarding to AWS.

1. Scope and design goals

This document defines the recommended implementation for a self-hosted garden weather station with no direct internet service at the garden.

Primary goals are: reliable local weather acquisition, low-power off-grid operation, deterministic interfaces between devices, and a simple bring-up path that lets each subsystem be tested independently before the full system is installed outdoors.

- Operate the garden electronics from a 12 V solar-battery system with a regulated 5 V distribution rail.
- Use the Tempest hub's local UDP broadcast rather than the Tempest cloud API for the garden node.
- Use Meshtastic over US915 LoRa as the backhaul link between the garden and the house.
- Use USB from the home Meshtastic node to a Raspberry Pi gateway at the house.
- Support staged testing: node-to-node radio first, mock weather payloads second, real Tempest packets third, AWS integration last.

2. Final recommended architecture

The system is intentionally split into two independent halves. The garden half is responsible for sensing, local packet collection, and radio uplink. The home half is responsible for packet reception, gateway processing, and onward delivery to AWS. This separation keeps the garden node simple and power-efficient while putting the internet-facing logic in the house where power and debugging access are easy.

Layer	Garden side	House side	Purpose
Weather sensing	Tempest outdoor sensor + Tempest hub	-	Generate and publish weather observations
Local network	Small Wi-Fi router / LAN	Home Wi-Fi optional	Provide local IP connectivity where needed
Garden controller	Raspberry Pi Pico W	-	Read local UDP data and package outbound weather messages
Radio backhaul	Garden Meshtastic node	Home Meshtastic node	LoRa relay between the two locations
Gateway	-	Raspberry Pi via USB to home node	Read received packets and forward/store them
Cloud / website	-	AWS services + website	Storage, API, and presentation

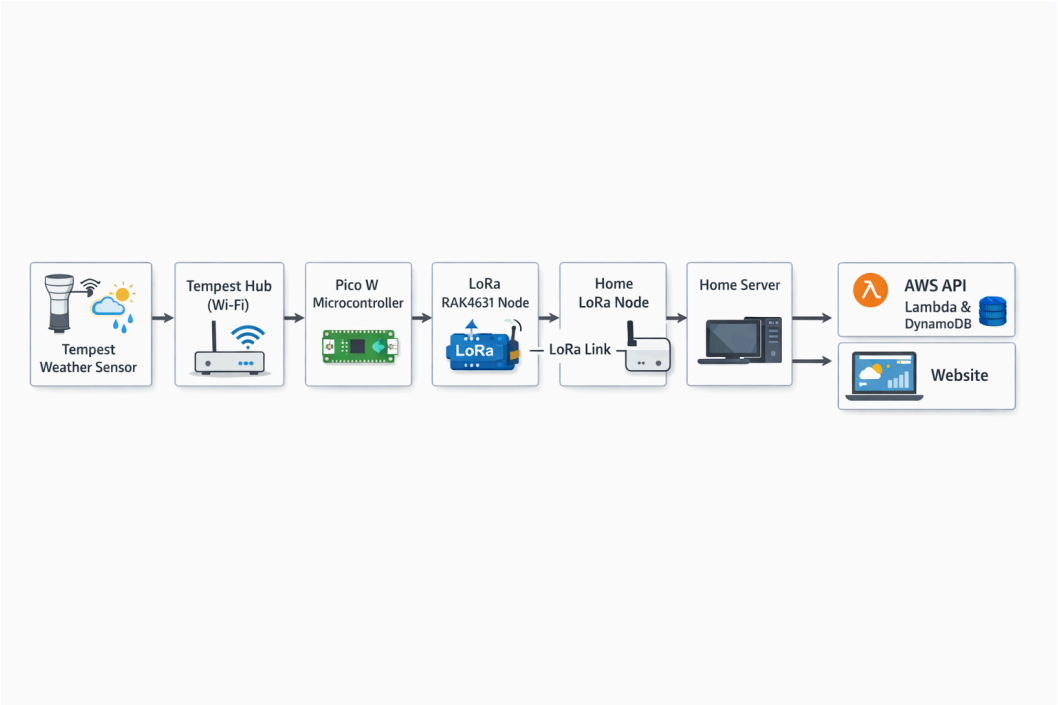

Figure 1. Recommended architecture from garden node to AWS-backed website.

3. Hardware bill of materials

The table below reflects the currently recommended hardware stack, including decisions made after reviewing the RAK starter kit contents and power-input options.

Subsystem	Part	Qty	Use in system	Notes
Weather station	WeatherFlow Tempest sensor + hub	1	Outdoor weather sensing and local hub broadcast	Hub joins local garden Wi-Fi and broadcasts UDP on the LAN.
Garden networking	Small 2.4 GHz travel router	1	Private LAN for Tempest hub and Pico W	No WAN connection required for local operation.
Garden controller	Raspberry Pi Pico W / WH	1	Listens for UDP broadcast and sends packets over UART	Onboard LED can be used as transmit indicator during debugging.
Garden radio	RAK WisBlock starter kit (US915)	1	Garden Meshtastic node	Use LoRa antenna on the LoRa port and BLE antenna on the BLE port.
House radio	RAK WisBlock starter kit (US915)	1	Home Meshtastic node	Connect to Raspberry Pi by USB for gateway operation.

Subsystem	Part	Qty	Use in system	Notes
Gateway	Raspberry Pi (house)	1	Home packet receiver / uploader	Runs Meshtastic Python listener and forwards data to AWS.
Power	12 V LiFePO4 battery	1	Primary energy storage at garden	Charged from solar controller.
Power	Solar panel + charge controller	1	Garden charging system	Maintain 12 V battery bus.
Power	12 V to 5 V buck converter	1	Garden low-voltage rail	Feeds Tempest hub, router, Pico, and RAK node via USB-C/5 V rail.
RF option	U.FL/MHF1 to RP-SMA pigtail + 902-928 MHz blade antenna	optional	Outdoor antenna upgrade	Replaces the stock LoRa antenna on the LoRa U.FL port for the final installation if more range margin is needed.

4. Antenna and connector notes for the RAK nodes

The RAK starter kit includes three relevant RF accessories observed during unboxing: a flat LoRa antenna labeled for 902-928 MHz, a BLE antenna, and a small SMA-style pigtail. These do not all connect at the same time.

- BLE antenna: connects to the BLE U.FL connector and is used for phone-app pairing and local Bluetooth configuration.
- Flat LoRa antenna: connects to the LoRa U.FL connector and is the default antenna for bench testing and initial bring-up.
- SMA pigtail: plugs into the same LoRa U.FL connector as the flat LoRa antenna and is used only when replacing the stock LoRa antenna with an external 902-928 MHz antenna.

Important: Only one antenna may be connected to the LoRa port at a time. For bench tests, keep the stock flat LoRa antenna installed. For the final outdoor build, remove the flat LoRa antenna, install the SMA pigtail on the LoRa port, and connect the external 902-928 MHz antenna to the pigtail.

5. Interface specification: every device-to-device link

This section explicitly defines how each device communicates with the device immediately before or after it. This is the critical systems-engineering view of the build and should be treated as the source of truth during implementation.

Link	Physical interface	Protocol	Direction	Key settings	Notes
Tempest sensor -> Tempest hub	Vendor wireless radio link	WeatherFlow internal link	Bidirectional vendor-managed	No user configuration beyond normal Tempest setup	Treat as appliance behavior; not

Link	Physical interface	Protocol	Direction	Key settings	Notes
					part of custom coding work.
Tempest hub <-> garden router	2.4 GHz Wi-Fi	IP over Wi-Fi	Hub joins LAN as client	Router provides local SSID and DHCP	Internet is not required for local UDP broadcast collection.
Pico W <-> garden router	2.4 GHz Wi-Fi	IP over Wi-Fi	Pico joins LAN as client	Same SSID/subnet as Tempest hub	Pico must be on the same local LAN as the hub to hear the UDP broadcasts.
Tempest hub -> Pico W	Local LAN	UDP broadcast, JSON payloads	One-way broadcast from hub	Listen on UDP port 50222; message types include obs_st and rapid_wind	Preferred source of weather data for the garden node. No cloud token required.
Pico W <-> garden Meshtastic node	3-wire UART + shared ground	UART serial	Primarily Pico TX -> node RX; optional reverse path	3.3 V logic, suggested 115200 baud, 8N1, connect TX<->RX and GND<->GND	Do not feed 5 V logic into the RAK UART pins. Power voltage and UART logic level are different concepts.
Garden Meshtastic node <-> home Meshtastic node	LoRa radio via antennas	Meshtastic over LoRa	Bidirectional	Region: United States / US915; same primary channel and PSK on both nodes	Bench testing confirmed message exchange once region/channel matched. Node list metadata may lag even when messages pass.
Home Meshtastic node <-> Raspberry Pi	USB-C cable	USB serial / Meshtastic host interface	Bidirectional	Use USB for both power and host communication	Recommended over Bluetooth for reliability and easier debugging.
Raspberry Pi <-> AWS	Home internet connection	HTTPS	Bidirectional request/response	POST latest weather payloads to API endpoint	Keep cloud auth and internet-facing logic on the house side, not in the garden node.

Link	Physical interface	Protocol	Direction	Key settings	Notes
Website <-> AWS API	Internet	HTTPS GET / JSON	Request/response	Simple polling is recommended initially	WebSockets can be added later if desired but are not required for MVP.

6. Data models and packet formats

6.1 Tempest UDP packets consumed by the Pico

The Pico should start by listening for any packet on UDP 50222, then narrow to the specific message types used by the application. For the MVP, the most important are the full observation packet and the rapid wind packet.

```
{
  "type": "obs_st",
  "obs": [[ epoch_s, wind_lull_ms, wind_avg_ms, wind_gust_ms, wind_dir_deg,
wind_sample_interval_s, pressure_hpa, temp_c, humidity_pct, illuminance_lux,
uv_index, solar_wm2, rain_mm, precip_type, lightning_km, lightning_count,
battery_v, report_interval_min ]]
}
```

Suggested MVP field extraction from obs_st: timestamp, average wind, wind direction, pressure, temperature, humidity, and rain amount. The rapid_wind packet can be used later if the site needs more frequent wind updates than the one-minute observation cadence.

6.2 Pico to Meshtastic payload format

During early bring-up, use short numbered plain-text messages such as pico-test-0001. Once the UART and radio chain is proven, switch to a compact JSON payload. Keep the payload small and stable so debugging on the home gateway remains easy.

```
{
  "msg_id": 12,
  "ts": 1772841600,
  "temp_c": 22.4,
  "hum": 55,
  "pres_hpa": 1012.7,
  "wind_ms": 3.1,
  "wind_dir": 245,
  "rain_mm": 0.0
}
```

Recommended UART framing rule for bench testing: one JSON object per line terminated by a newline. This makes it easy to print, log, and replay traffic during debugging.

6.3 Home gateway parsing model

The Raspberry Pi should treat the home Meshtastic node as the source of received packets. The gateway service should log raw packets first, then parse JSON second, then hand the parsed object to storage or AWS upload logic. This layered approach makes field-level bugs easier to isolate.

7. Power architecture

The garden node should be designed around a primary 12 V solar battery bus, with a regulated 5 V rail created by a buck converter. This matches the actual power requirements discovered during hardware review: the RAK board can be powered from USB-C, the Tempest hub is a 5 V device, and the router is a 5 V device. The RAK board's onboard solar and battery JST connectors are useful for tiny standalone Meshtastic nodes but are not the best center of power management for the larger garden system.

```
12 V solar panel -> charge controller -> 12 V battery bus
                                |
                                +-> 12 V to 5 V buck converter -> 5 V distribution
rail
                                                                  |- Tempest hub
                                                                  |- garden router
                                                                  |- Pico W
                                                                  `- garden RAK node
via USB-C
```

Garden load	Nominal input	Recommended feed	Notes	Approx. daily energy
Tempest hub	5 V	5 V regulated rail	Treat as a normal USB-powered device.	14.4 Wh/day (planning value)
Travel router	5 V	5 V regulated rail	Use a stable converter with headroom.	36 Wh/day (planning value)
Pico W	5 V USB or regulated feed	5 V regulated rail	GPIO logic remains 3.3 V.	6 Wh/day (planning value)
Garden RAK node	5 V via USB-C	5 V regulated rail	Preferred over using the onboard 3.7 V battery charger in this system.	0.6 Wh/day (planning value)
Total garden electronics	-	12 V bus through one buck converter	Size battery and panel for margin and cloudy days.	~57 Wh/day

A practical installation target is a single high-quality 5 V buck converter with enough current margin for all 5 V garden electronics together. A 5 V / 5 A converter is a comfortable starting point for the garden enclosure.

8. Home gateway design

The home node should connect to a Raspberry Pi by USB-C. This is the recommended permanent interface for the house-side gateway because it provides power and a stable host connection at the same time. Bluetooth should remain available for app-based configuration, but USB is the right operational link for a long-running receiver service.

- Use the Meshtastic app only for initial node setup and troubleshooting.
- Use the Raspberry Pi as the host that reads the home node over USB.

- Run a small Python service on the Raspberry Pi that logs received packets, parses weather payloads, and writes the latest reading to local storage before uploading to AWS.
- Do not rely on the app's Nodes list as the only health signal. Bench testing showed that messages can pass even while node metadata or displayed names lag behind.

9. Bench bring-up plan before adding the Pico

Bench bring-up is now partially complete: both RAK nodes have been configured with the United States region, a matching private channel, and message passing was confirmed from home-node to garden-node. The next steps below formalize the recommended test sequence going forward.

Phase	Goal	Action	Pass condition
A	Node setup	Connect each node in the Meshtastic app, set region, assign friendly names, confirm channel/PSK match.	Both nodes save config and remain reachable from the app.
B	Radio proof	Send short messages both ways between the two RAK nodes.	Both directions succeed even if the Nodes tab still shows cached strangers or fallback IDs.
C	Host gateway proof	Connect home node to Raspberry Pi by USB and verify the Pi can see the node.	Pi detects the node and a listener script can log inbound test messages.
D	Mock payload proof	Inject numbered fake weather messages before introducing real Tempest packets.	Gateway receives and parses stable mock payloads.
E	Local weather input	Add the Pico and Tempest hub/router to replace the mock payload generator.	Real UDP weather fields arrive through the same end-to-end path.
F	Cloud handoff	Add AWS upload and website retrieval last.	Gateway can post latest observation to AWS and the website can retrieve it.

10. Pico integration plan

Start with mocked traffic before attempting Tempest integration. This is the lowest-risk path because it proves the UART framing and the radio chain independently of the weather source.

- Phase 1: make the Pico print numbered test strings over USB serial only.
- Phase 2: make the Pico send the same numbered strings over UART to the garden RAK node.
- Phase 3: add an LED blink on the Pico each time it attempts transmission.
- Phase 4: replace the text strings with compact JSON mock payloads.
- Phase 5: connect the Pico to the garden router, listen on UDP 50222, and replace mock payloads with real Tempest fields.

The onboard Pico LED is sufficient for the first visible transmit indicator. A short blink after every UART write is recommended so the bench operator has an immediate local sign that the Pico code reached its transmit step.

11. Known implementation notes from the current bench test

- Meshtastic app permissions on Android can block discovery if Bluetooth / nearby-device permissions are not granted.
- Saving node configuration can appear stuck at 0% if the Bluetooth link hangs; cancel, reconnect, and retry rather than waiting indefinitely.
- The app's Nodes list can show previously discovered nodes or incomplete metadata even when the current private channel is correct.
- Friendly node names may lag behind and fall back to a short identifier until node metadata fully refreshes.
- Actual message exchange is a more trustworthy radio-health signal than the Nodes list during early bring-up.

12. AWS side (intentionally lightweight for now)

12.1 Backend Overview

The weather station cloud backend should use a lightweight, low-cost, secure serverless architecture in AWS. The system is not intended to be public-facing at scale and is expected to have only one or two trusted clients, primarily the home Raspberry Pi and optionally a website or dashboard for viewing current conditions and history.

The recommended architecture is:

AWS API Gateway (HTTP API) → AWS Lambda → Amazon DynamoDB

This design provides the following benefits:

- Very low monthly operating cost at the expected traffic volume
- Good performance for small payloads and simple reads/writes
- Strong security using AWS IAM for authenticated write access
- Minimal operational overhead
  Easy deployment and repeatability using CloudFormation templates
- Easy future expansion for alerts, dashboards, or analytics

This architecture is preferred over a traditional EC2-hosted API because the workload is extremely small, intermittent, and predictable. It is also simpler and cheaper than using AWS IoT Core for this initial version

12.2 Functional Requirements

The cloud API must support two primary operations:

1. **Post weather observations**

    - The home Raspberry Pi sends weather data to AWS over HTTPS
    - Each payload contains a timestamp and a small set of current weather values

    - The API validates and stores the observation in DynamoDB

2. **Fetch weather observations**

    - A website, dashboard, or trusted client can retrieve:

        - the most recent observation
        - historical observations for a time range

The initial system should optimize for:

- reliable ingest
- simple retrieval of latest data
- low cost
- straightforward security

12.3 Recommended AWS Services

1. API Gateway HTTP API

API Gateway should expose the HTTPS endpoints used by the Raspberry Pi and viewer clients.

Recommended routes:

- `POST /observations`
- `GET /observations/latest`
- `GET /observations?from=...&to=...`

Why HTTP API:

- lower cost than REST API
- lower latency
- sufficient for this use case
- supports IAM authorization for write endpoints

2. AWS Lambda

Lambda functions should implement the API logic.

Recommended functions:

- **Ingest function**

    - accepts posted weather data
    - validates schema
    - writes to DynamoDB
    - updates the “latest” record

- **Read function**

    - fetches the latest record

    - optionally queries a historical time range

Lambda is appropriate because:

- requests are infrequent
- compute needs are tiny
- no server maintenance is required
- it integrates cleanly with API Gateway and DynamoDB

3. Amazon DynamoDB

DynamoDB should store both time-series weather data and the latest reading.

Recommended table design:

- **Table name:** `WeatherObservations`
- **Partition key:** `stationId`
- **Sort key:** `timestamp`

This allows efficient retrieval of all readings for a station over time.

A second pattern should also be used for fast current reads:

- store a special item such as `recordType = latest`
- or use a separate “latest” table if preferred

For this project, a single table with a special latest item is usually sufficient and simpler.

4. CloudWatch

CloudWatch should be used for:

- Lambda logs
- basic API monitoring
- optional alarms for failures or elevated error rates

Logging should be kept minimal to control cost.

13. Open items and recommended next actions

1. Connect the home RAK node to a Raspberry Pi over USB and verify the Pi can enumerate and communicate with it.
2. Receive the Pico and bench-test UART-only mock messages before adding the Tempest hub.
3. Finalize the garden 12 V to 5 V distribution layout and choose the exact buck converter and enclosure.
4. Decide later whether the outdoor node needs the optional external 902-928 MHz blade antenna or whether the stock LoRa antenna is sufficient.